n-body (plaquette) terms in ALPS, step by step

ALPS models had two kinds of terms: SITETERM (one site) and BONDTERM (two sites). This branch adds a third, PLAQUETTETERM: one operator expression on an ordered group of any number of sites, attached to a new <PLAQUETTE> element of the lattice. A molecular Hamiltonian's (pq|rs), a ring exchange or a correlated pair hop can now be written down exactly, with nothing truncated. The terms are used by exact diagonalization (`fulldiag`, `sparsediag`).

This document follows one small problem through ALPS: the input files, the plaquette's local table, the fill of the many-body matrix, and the eigenvalues. It is the same system as quartic_ED_walkthrough.pdf (one 2×2 plaquette, t = 1, U = 4, K = 0.5, one ↑ and one ↓ electron). Every matrix and number below was printed by ALPS itself (`pdf_source/`).

Optional: nothing changes unless the input asks for it

There is no switch to turn on and nothing to change in existing inputs. The new code runs only when the lattice contains a <PLAQUETTE> and the model contains a <PLAQUETTETERM>:

lattice has <PLAQUETTE>	model has <PLAQUETTETERM>	what happens	checked
no	no	exactly as before	stock models: master vs branch bit-identical; full ctest 931/931
yes	no	plaquettes are ignored	stock 'fermion Hubbard' on this lattice: E0 = -3.4185507189
no	yes	term has nothing to act on (like a BONDTERM on a graph without edges)	K = 0.5 model, plaquette removed: E0 = -3.4185507189 (= the K = 0 value)
yes	yes	fulldiag / sparsediag add the term; DMRG and QMC codes stop with a message	this document; dmrg refuses (below)

The refusal, so a model is never silently truncated: The model has <PLAQUETTETERM>s (n-site terms) and the lattice has <PLAQUETTE>s, but this application supports only site and bond terms. n-site terms are currently supported by exact diagonalization (fulldiag, sparsediag).

Step 1 — Lattice: a <PLAQUETTE> is an ordered group of sites

A <GRAPH> gets plaquette elements next to its edges. Each <SITE id> is a vertex id (1-based, like EDGE source/target). The order of the sites is what the model's role names bind to: here i, j, k, l = vertices 1, 2, 4, 3, i.e. ALPS sites 0, 1, 3, 2 going round the square. The plaquette is stored in its own list beside the graph; it adds no bonds.

```
<LATTICES>
  <GRAPH name="2x2 plaquette" vertices="4">
    <!--  3 ---- 4
          |      |
          1 ---- 2      (ALPS numbers sites 0..3 internally) -->
    <EDGE source="1" target="2"/>
    <EDGE source="3" target="4"/>
    <EDGE source="1" target="3"/>
    <EDGE source="2" target="4"/>
    <!-- new: an ordered group of sites; the order fixes the roles i j k l -->
    <PLAQUETTE type="0">
      <SITE id="1"/><SITE id="2"/><SITE id="4"/><SITE id="3"/>
    </PLAQUETTE>
  </GRAPH>
</LATTICES>
```

```
ALPS builds:

  sites      0 1 2 3
  bonds      (0,1) (2,3) (0,2) (1,3)
  plaquettes type 0: (0, 1, 3, 2)

     2 ----- 3
     |   K   |
     0 ----- 1

  roles  i=0  j=1  k=3  l=2
```

For a periodic lattice the plaquette goes in the <UNITCELL>, with each site given as vertex + cell offset exactly like an EDGE's SOURCE/TARGET. ALPS repeats it over the lattice, wraps it at periodic boundaries and drops it where a site would fall off an open boundary:

```
<UNITCELL name="ladder cell" dimension="1" vertices="2">
  ...edges as usual...
  <PLAQUETTE type="0">
    <SITE vertex="1" offset="0"/> <SITE vertex="2" offset="0"/>
    <SITE vertex="2" offset="1"/> <SITE vertex="1" offset="1"/>
  </PLAQUETTE>
</UNITCELL>
```

Step 2 — Model: a <PLAQUETTETERM> is written like a BONDTERM

`sites="i j k l"` names the roles; the term is any expression in the site operators (and library operators), on as many roles as the plaquette has sites. As for bond terms, `type="…"` restricts a term to one plaquette type, and without it `#` in parameter names is replaced by the type (K# → K0, K1, …). Fermion signs follow the operator order as written. The SITEBASIS and BASIS at the top of model.xml are the usual fermion ones; only the HAMILTONIAN is shown.

```
<HAMILTONIAN name="Hubbard with pair hopping">
  <PARAMETER name="t" default="1"/>
  <PARAMETER name="U" default="0"/>
  <PARAMETER name="K" default="0"/>
  <BASIS ref="fermion"/>
  <SITETERM site="i">U*n_up(i)*n_down(i)</SITETERM>
  <BONDTERM source="i" target="j">
    -t*(cdag_up(i)*c_up(j) + cdag_up(j)*c_up(i) + cdag_down(i)*c_down(j) + cdag_down(j)*c_down(i))
  </BONDTERM>
  <!-- new: one term on four sites -->
  <PLAQUETTETERM sites="i j k l">
    K*(cdag_up(l)*cdag_down(k)*c_down(j)*c_up(i) + cdag_up(i)*cdag_down(j)*c_down(k)*c_up(l))
  </PLAQUETTETERM>
</HAMILTONIAN>
```

The K term is quartic on four different sites: it moves an ↑ electron from i to l and a ↓ electron from j to k together (and back). No site or bond term can produce it.

Step 3 — Run it like any other ALPS model

```
LATTICE_LIBRARY = "lattice.xml"
MODEL_LIBRARY   = "model.xml"
GRAPH           = "2x2 plaquette"
MODEL           = "Hubbard with pair hopping"
CONSERVED_QUANTUMNUMBERS = "Nup,Ndown"
Nup_total   = 1
Ndown_total = 1
t = 1
U = 4
{ K = 0 }
{ K = 0.5 }
```

```
$ parameter2xml parm
$ fulldiag parm.in.xml

lowest levels (fulldiag, N↑ = N↓ = 1):
  K = 0    -3.418551  -2.000000  -2.000000  -0.828427
  K = 0.5  -3.359701  -2.000000  -1.882634  -1.106328
```

All 16 levels for K = 0.5: -3.359701 -2.000000 -1.882634 -1.106328 -0.828427 -0.188513 -0.000000 +0.000000 +0.123791 +1.630788 +2.000000 +2.144396 +4.000000 +4.794841 +4.828427 +5.843361. These are the numbers in quartic_ED_walkthrough.pdf (E0 = −3.3597011665, and −3.4185507189 at K = 0).

Step 4 — The ALPS basis of the (N↑ = 1, N↓ = 1) sector

Each site has 4 local states, numbered by ALPS from the quantum numbers (Nup, Ndown): 0 = · (0, 0), 1 = ↓ (0, 1), 2 = ↑ (1, 0), 3 = ↕ (1, 1) (· empty, ↕ doubly occupied). A basis state is the tuple of local states on sites 0…3, and ALPS sorts the 16 tuples that satisfy the constraints. The Jordan–Wigner order is site 0, 1, 2, 3 and ↑ before ↓ on a site, i.e. mode = 2·site + spin as in the earlier PDFs; the bit column is that occupation string.

ALPS index	local states (site 0 1 2 3)	bits m7…m0	electrons	picture (2 3 / 0 1)
0	0 0 0 3	11000000	|↑3 ↓3>	· ↕ / · ·
1	0 0 1 2	01100000	|↓2 ↑3>	↓ ↑ / · ·
2	0 0 2 1	10010000	|↑2 ↓3>	↑ ↓ / · ·
3	0 0 3 0	00110000	|↑2 ↓2>	↕ · / · ·
4	0 1 0 2	01001000	|↓1 ↑3>	· ↑ / · ↓
5	0 1 2 0	00011000	|↓1 ↑2>	↑ · / · ↓
6	0 2 0 1	10000100	|↑1 ↓3>	· ↓ / · ↑
7	0 2 1 0	00100100	|↑1 ↓2>	↓ · / · ↑
8	0 3 0 0	00001100	|↑1 ↓1>	· · / · ↕
9	1 0 0 2	01000010	|↓0 ↑3>	· ↑ / ↓ ·
10	1 0 2 0	00010010	|↓0 ↑2>	↑ · / ↓ ·
11	1 2 0 0	00000110	|↓0 ↑1>	· · / ↓ ↑
12	2 0 0 1	10000001	|↑0 ↓3>	· ↓ / ↑ ·
13	2 0 1 0	00100001	|↑0 ↓2>	↓ · / ↑ ·
14	2 1 0 0	00001001	|↑0 ↓1>	· · / ↑ ↓
15	3 0 0 0	00000011	|↑0 ↓0>	· · / ↕ ·

Orange: the two states the K term connects.

Step 5 — The plaquette's local table (NSiteOperator::matrix)

ALPS first turns the term string into a table on the 4 plaquette sites alone. It evaluates the expression on every tuple of local states in role order (i, j, k, l): $4^4 = 256$ input tuples. The table is stored sparsely, and only 32 (input, output) pairs are nonzero. The site operators act right to left; each odd (fermionic) operator is moved into role order $O_i\, O_j\, O_k\, O_l$, with one sign for each odd operator it passes. Signs between the plaquette and the rest of the lattice are not in the table; they are added in Step 6, as for bond terms. The table depends only on the plaquette type and its site types, so it is built once and reused on every plaquette of the lattice.

Below: the 16 entries for the first half, $K\, c\dagger_{l\uparrow}\, c\dagger_{k\downarrow}\, c_{j\downarrow}\, c_{i\uparrow}$, in units of K. The other 16 (the Hermitian conjugate) are the same with input and output swapped. There are 16 because each site may also hold the other spin as a spectator (2^4). The sign depends only on those spectators: $c_\downarrow$ acting on ↕ passes the ↑ on the same site (↑ comes first), which gives −1, while $c_\uparrow$ on ↕ gives +1.

in (i j k l)		out (i j k l)		value / K	spectators
2100	↑ ↓ · ·	0012	· · ↓ ↑	+1	none
2101	↑ ↓ · ↓	0013	· · ↓ ↕	+1	l
2120	↑ ↓ ↑ ·	0032	· · ↕ ↑	-1	k
2121	↑ ↓ ↑ ↓	0033	· · ↕ ↕	-1	k, l
2300	↑ ↕ · ·	0212	· ↑ ↓ ↑	-1	j
2301	↑ ↕ · ↓	0213	· ↑ ↓ ↕	-1	j, l
2320	↑ ↕ ↑ ·	0232	· ↑ ↕ ↑	+1	j, k
2321	↑ ↕ ↑ ↓	0233	· ↑ ↕ ↕	+1	j, k, l
3100	↕ ↓ · ·	1012	↓ · ↓ ↑	+1	i
3101	↕ ↓ · ↓	1013	↓ · ↓ ↕	+1	i, l
3120	↕ ↓ ↑ ·	1032	↓ · ↕ ↑	-1	i, k
3121	↕ ↓ ↑ ↓	1033	↓ · ↕ ↕	-1	i, k, l
3300	↕ ↕ · ·	1212	↓ ↑ ↓ ↑	-1	i, j
3301	↕ ↕ · ↓	1213	↓ ↑ ↓ ↕	-1	i, j, l
3320	↕ ↕ ↑ ·	1232	↓ ↑ ↕ ↑	+1	i, j, k
3321	↕ ↕ ↑ ↓	1233	↓ ↑ ↕ ↕	+1	i, j, k, l

Step 6 — Fill the many-body matrix (hamiltonian_matrix::apply_operator)

For every basis state, ALPS reads the local states on the plaquette's sites (in role order), looks up that row of the table, and writes each output tuple back into the state to get the target basis state. The only extra ingredient is the Jordan–Wigner sign of each odd operator O_r: $(-1)^n$, where n is the number of fermions on sites before site(r), counted in the state as already changed by the operators applied before it. For two sites this is exactly the rule ALPS already uses for bond terms. One column, worked through:

```
column 14 = |↑0 ↓1>  (picture · · / ↑ ↓),  plaquette tuple (i j k l) = 2 1 0 0
table entry -> output tuple 0 0 1 2, value +1 K
operators in role order, applied last role first:
  role l = site 2: · -> ↑   fermions on sites < 2: 2  -> even, +1
  role k = site 3: · -> ↓   fermions on sites < 3: 2 (+1: role l already applied on site 2)  -> odd, -1
  role j = site 1: ↓ -> ·   fermions on sites < 1: 1  -> odd, -1
  role i = site 0: ↑ -> ·   fermions on sites < 0: 0  -> even, +1
result: sign +1, row 2 = |↑2 ↓3>,  H[2,14] += +0.5
```

This is the entry the earlier PDF finds as V[4,7|0,3] → H[11,1] = +K in its own state numbering. Re-implementing this rule in numpy (make_pdf.py, `fill`) on the 32 table entries reproduces ALPS's K contribution exactly: max |difference| = 0e+00.

Step 7 — The full 16 × 16 Hamiltonian, as ALPS builds it

Rows and columns use the ALPS basis of Step 4. Yellow: U on doubly occupied sites (site terms). Orange: the K term (plaquette term). Everything else: hopping (bond terms), whose ± sign comes from the same Jordan–Wigner rule.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	+4	+1	-1	·	+1	·	-1	·	·	·	·	·	·	·	·	·
1	+1	·	·	+1	·	·	·	+1	·	-1	·	·	·	·	·	·
2	-1	·	·	-1	·	+1	·	·	·	·	·	·	-1	·	+0.5	·
3	·	+1	-1	+4	·	·	·	·	·	·	+1	·	·	-1	·	·
4	+1	·	·	·	·	-1	·	·	+1	-1	·	·	·	·	·	·
5	·	·	+1	·	-1	·	·	·	·	·	-1	·	·	·	+1	·
6	-1	·	·	·	·	·	·	-1	-1	·	·	·	-1	·	·	·
7	·	+1	·	·	·	·	-1	·	·	·	·	+1	·	-1	·	·
8	·	·	·	·	+1	·	-1	·	+4	·	·	+1	·	·	-1	·
9	·	-1	·	·	-1	·	·	·	·	·	-1	-1	·	·	·	·
10	·	·	·	+1	·	-1	·	·	·	-1	·	·	·	·	·	+1
11	·	·	·	·	·	·	·	+1	+1	-1	·	·	·	·	·	+1
12	·	·	-1	·	·	·	-1	·	·	·	·	·	·	-1	-1	·
13	·	·	·	-1	·	·	·	-1	·	·	·	·	-1	·	·	-1
14	·	·	+0.5	·	·	+1	·	·	-1	·	·	·	-1	·	·	-1
15	·	·	·	·	·	·	·	·	·	·	+1	+1	·	-1	-1	+4

Check against the independent prototype. Re-indexed into the earlier PDF's state order, ALPS's matrix equals the prototype's (hyper_ed.py: normal-ordered coefficient tables and bit-string fill, no ALPS code) entry by entry, signs included: max |difference| = 0.0. Eigenvalues: E0 = -3.3597011665 (K = 0.5), -3.4185507189 (K = 0).

Step 8 — Benchmarks

(a) Molecules: ALPS ED vs QDK CASCI, full (pq|rs). A molecular Hamiltonian's two-body part touches 3 and 4 distinct orbitals, so before this change it could not be written for ALPS without truncation. Now every orbital set touched by a nonzero (pq|rs) becomes a <PLAQUETTE> with its own <PLAQUETTETERM>. All orbitals active, STO-3G. The last column compares the whole spectrum with a dense Jordan–Wigner diagonalization of the same integrals (numpy, independent of both codes). Result: ALL AGREE.

molecule	sector (N↑,N↓)	plaquettes 2/3/4-site	QDK CASCI	ALPS ED	|ΔE|	spectrum vs dense JW
H4 chain 0.9 A	(2,2)	6/4/1	-2.1803166186	-2.1803166186	1.8e-15	7.1e-15
H4 chain 0.9 A	(2,1)	6/4/1	-1.7659014705	-1.7659014705	4.4e-15	3.6e-15
H4 chain 0.9 A	(3,1)	6/4/1	-1.8916100906	-1.8916100906	2.7e-15	2.7e-15
H4 rectangle 0.8x1.2 A	(2,2)	6/0/1	-2.0922392274	-2.0922392274	8.9e-16	9.3e-15
H4 rectangle 0.8x1.2 A	(3,1)	6/0/1	-1.8188072640	-1.8188072640	3.6e-15	6.2e-15
H4 bent/asym	(2,2)	6/4/1	-2.1573698719	-2.1573698719	7.1e-15	5.3e-15
H4 bent/asym	(2,1)	6/4/1	-1.7521256324	-1.7521256324	4.4e-15	4.0e-15
H4 bent/asym	(1,1)	6/4/1	-0.9132368580	-0.9132368580	1.8e-15	4.0e-15
H3+ triangle	(1,1)	3/1/0	-1.2693966626	-1.2693966626	4.4e-16	4.0e-15
H3+ triangle	(2,0)	3/1/0	-0.6869268798	-0.6869268798	0.0e+00	8.9e-16
H4 chain 1.5 A (stretched)	(2,2)	6/4/1	-1.9961503613	-1.9961503613	2.7e-15	4.9e-15
H4 chain 1.5 A (stretched)	(3,1)	6/4/1	-1.9255585303	-1.9255585303	1.8e-15	4.4e-15
LiH 1.6 A (6 orbitals)	(2,2)	15/16/1	-7.8823243948	-7.8823243948	5.3e-15	3.9e-14
LiH 1.6 A (6 orbitals)	(2,1)	15/16/1	-7.6141564878	-7.6141564878	7.1e-15	1.6e-14
H6 ring 1.0 A	(3,3)	15/14/9	-3.2374767382	-3.2374767382	1.8e-15	4.1e-14

(b) Exact answer on 4 sites. Open 4-site chain plus U on one delocalized hopping eigenmode $a = \Sigma\, \varphi_i\, c_i$: $H = -t\,\Sigma$ hop $+ U\, n_{a\uparrow}\, n_{a\downarrow}$. Written in the site basis, the U term is one PLAQUETTETERM with 1-, 2-, 3- and 4-site pieces. The spectrum is known exactly: $\Sigma\, \varepsilon_{\text{occupied}} + U\cdot$[mode a doubly occupied]. Checked for two modes, three role orderings and five sectors. A term that changes N by 4 is checked in the full 256-state space against a dense matrix, and with N↑, N↓ conserved it is refused. 32 runs; ALL OK.

(c) Ladders vs the prototype. 2×L Hubbard ladders with a pair-hopping plaquette term, open and periodic, several sectors: ALL OK.

(d) Random two-body terms on 2- and 3-site plaquettes, all index patterns, with interleaved site orders, vs a dense Jordan–Wigner matrix: ALL OK.

(e) Unit tests. src/alps/model/tests/nsite_terms.cpp (GoogleTest): unit-cell plaquettes at open and periodic boundaries, GRAPH plaquettes, invalid plaquettes, ladder reference energies, K = 0 equals the plain model, the analytic case above, sector violation, typed and # terms, parameter precision, and the refusal in other applications. Full ctest: 931/931 pass.

Raw output and the scripts are in `benchmarks/`.

Step 9 — What changed in ALPS

file	change
lattice/plaquette.h (new)	plaquette {type, sites}; reading <PLAQUETTE> in GRAPH and UNITCELL
lattice/graph.h, latticelibrary.h/.C	GRAPH accepts <PLAQUETTE>; the library keeps them per graph
lattice/unitcell.h/.C	UNITCELL accepts <PLAQUETTE> (vertex + offset), and writes it back
lattice/latticegraph.h	unit-cell plaquettes repeated over the lattice, same boundary rules as edges
lattice/graph_helper.h	plaquettes(), num_plaquettes()
model/nsiteoperator.h (new)	NSiteOperator, its sign-correct evaluator, sparse local table
model/nsiteterm.C (new)	read/write <PLAQUETTETERM>
model/globaloperator.h/.C, default_term.h	HAMILTONIAN holds plaquette terms (typed and #)
model/hamiltonian_matrix.hpp	plaquette loop: local table per type, Jordan–Wigner fill, sector check
model/model_helper.h	applications without support refuse plaquette terms instead of dropping them
model/siteterm.C, bondterm.C	fix: parameters were rounded to 6 digits (see below)
model/tests/nsite_terms.cpp (new)	unit tests
CHANGELOG.md, model/docs/	changelog entry, syntax reference (plaquette_terms.md), this folder

One fix to existing behaviour. SITETERM and BONDTERM substituted parameter values into the expression with 6 significant digits, so `t0=0.123456789` acted as 0.123457. Molecular integrals exposed this as a ~$5 \cdot 10^{-6}$ error against CASCI. Values are now written with full double precision. Inputs with short parameter values (t = 1, U = 4, …) give bit-identical results; longer values are now used exactly as given.

Limits. Only exact diagonalization supports plaquette terms for now; DMRG and the QMC codes refuse them. Building the local table goes through ALPS's expression evaluator, which is slow for large molecular terms (~30 s for H6) but is done once per plaquette type. Plaquettes in a <GRAPH> are not yet written back out when ALPS writes a lattice library to XML.

Appendix — the fill (Step 6) in ALPS

src/alps/model/include/alps/model/hamiltonian_matrix.hpp:

```
// Fill one n-site term on plaquette p. The local table (see nsiteoperator.h)
// stores each product reordered as O_0 O_1 ... O_{n-1} (role order). Acting on
// a basis state, O_{n-1} is applied first; each odd O_r picks up the
// Jordan-Wigner sign (-1)^(number of fermions on sites before p.sites[r]),
// counted in the state as modified by the operators already applied. This is
// the rule the bond fill below encodes in closed form for n = 2.
template <class M, class G> template <class STATES, class V, class W>
void hamiltonian_matrix<M,G>::apply_operator(const STATES& states, const nsite_matrix<value_type>& mat,
      const plaquette& p, const V& x, W& y) const
{
  const std::size_t n = p.sites.size();
  const std::size_t nsites = graph_.num_sites();
  // fermion parity of every local state of every site
  std::vector<std::vector<bool> > parity(nsites);
  for (std::size_t s=0;s<nsites;++s)
    for (std::size_t k=0;k<basis_[s].size();++k)
      parity[s].push_back(is_fermionic(model_.model().basis().site_basis(graph_.site_type(s)),basis_[s][k]));

  std::vector<std::size_t> in(n), out(n);
  std::vector<bool> prefix(nsites+1);
  for (std::size_t i=0;i<dimension();++i) {         // loop over source states
    state_type state=states[i];
    for (std::size_t r=0;r<n;++r)
      in[r]=state[p.sites[r]];
    const typename nsite_matrix<value_type>::row_type& row = mat.rows[mat.index(in)];
    if (row.empty())
      continue;
    prefix[0]=false;                                // parity of sites < s
    for (std::size_t s=0;s<nsites;++s)
      prefix[s+1] = prefix[s] != parity[s][state[s]];
    for (std::size_t e=0;e<row.size();++e) {
      mat.decode(row[e].first,out);
      state_type newstate=state;
      for (std::size_t r=0;r<n;++r)
        newstate[p.sites[r]]=out[r];
      std::complex<double> phase;
      std::size_t j;
      boost::tie(j,phase) = states.index_and_phase(newstate);
      if (j>=dimension()) {
        if (uses_translation_invariance())
          continue;
        std::ostringstream msg;
        msg << "<PLAQUETTETERM> on plaquette of type " << p.type << " (sites";
        for (std::size_t r=0;r<n;++r)
          msg << " " << p.sites[r];
        msg << ") maps a basis state out of the basis: the term does not conserve "
               "the quantum numbers in CONSERVED_QUANTUMNUMBERS. Remove them from "
               "CONSERVED_QUANTUMNUMBERS or fix the term.";
        boost::throw_exception(std::runtime_error(msg.str()));
      }
      bool f=false;
      for (std::size_t r=n;r-->0;) {
        std::size_t s=p.sites[r];
        if (parity[s][in[r]]!=parity[s][out[r]]) {  // O_r is odd
          bool before=prefix[s];
          for (std::size_t q=r+1;q<n;++q)           // already applied O_q
            if (p.sites[q]<s && parity[p.sites[q]][in[q]]!=parity[p.sites[q]][out[q]])
              before=!before;
          if (before)
            f=!f;
        }
      }
      value_type val=row[e].second;
      if (f)
        val=-val;
      val *= numeric_cast<value_type>(phase * states.normalization(j)/states.normalization(i));
      y[i] += val*x[j];                 // set matrix element
      simplify(y[i]);               // optionally simplify a symbolic expression
    }
  }
}
```

The sign used when building the local table (Step 5), src/alps/model/include/alps/model/nsiteoperator.h:

```
// Operators are applied right to left (OperatorEvaluator::direction), so all
// operators already applied stand to the right of the current one. Moving the
// current operator on role r into role order means moving it right past the
// already-applied operators on roles q < r: one sign per odd one among them.
// For n = 2 this is BondOperatorEvaluator's rule with the roles swapped.
template <class I, class T, class STATE>
expression::Expression<T> NSiteOperatorEvaluator<I, T, STATE>::partial_evaluate_function(const std::string& name,
      const expression::Expression<T>& arg, bool isarg) const
{
  if (has_operator(name, arg)) {
    int r = role(arg);
```

```
    bool f = site_[r].fermionic();
    expression::Expression<T> e = site_[r].partial_evaluate_function(name, site_[r].site(), isarg);
    if (f != site_[r].fermionic() && numeric::is_nonzero(e)) {
      bool odd = false;
      for (int q = 0; q < r; ++q)
        if (site_[q].fermionic())
          odd = !odd;
      if (odd)
        e.negate();
    }
    return e;
  }
  return expression::ParameterEvaluator<T>(*this).partial_evaluate_function(name, arg, isarg);
}
```